R for Publication

Reporting, Tables, and Figures for Academic Manuscripts

Daniela Palleschi

2026-03-24

Table of contents

Preface **1**
Who this is for . . . 1
What you will learn . . . 1
Sessions . . . 1
How to use this book . . . 2
Required software . . . 2
Installing R, RStudio, and Quarto . . . 2
Installing TinyTeX . . . 2
Checking your versions . . . 3
Updating . . . 3
Packages . . . 3
Disclaimer . . . 4

Session 1: Quarto foundations **5**

(R)Project Hygiene **7**
Why project hygiene matters . . . 7
RProjects . . . 8
Creating a new RProject . . . 8
Folder structure . . . 8
Project-relative file paths . . . 10
renv: reproducible package management . . . 10
An alternative: pacman . . . 11
Quarto in an RProject . . . 11
Chapter slides . . . 12

Writing in Quarto **13**
Purpose . . . 13
Text formatting . . . 13
APA 7 statistical symbols . . . 14
Headers . . . 15
Lists and callouts . . . 15
Equations . . . 16
Cross-references . . . 17
Sections . . . 17

Numbered example sentences . . . 18
Citations . . . 18
Citation syntax . . . 19
Getting BibTeX entries . . . 19
References section . . . 20
Inline reporting . . . 20
Rounding and formatting . . . 20
Connecting to models . . . 21

Output Formats **23**
Purpose . . . 23
Rendering formats . . . 24
HTML . . . 24
Word . . . 25
PDF . . . 26
Format comparison . . . 27
APA 7 and `.tex` output . . . 27
Chapter slides . . . 28

Data and Tables **29**
Purpose . . . 29
Code chunks . . . 29
Chunk options . . . 30
Chunk labels . . . 31
Loading data . . . 31
Saving processed data . . . 32
Inline reporting . . . 32
Chapter slides . . . 33

Session 2: Model Outputs & Reporting **35**

Exercises **37**

Set-up **39**
Before you begin . . . 39
Option A: Use the workshop repository (recommended) . . . 39
Option B: Set up your own RProject . . . 40

Session 1 Exercises **41**
Writing in Quarto . . . 41
Exercise 1: Text formatting . . . 41
Exercise 2: Lists and callouts . . . 41
Exercise 3: Headers . . . 41

Exercise 4: Cross-references 41
Exercise 5: Citations 42
Exercise 6: References section 42
Exercise 7: Inline arithmetic 42
Output formats 42
Exercise 1: HTML with TOC 42
Exercise 2: Word output 42
Exercise 3: PDF and `.tex` output (stretch) 43
Data and tables 43
Exercise 1: A simple code chunk 43
Exercise 2: Echo without eval 44
Exercise 3: Silent chunk 44
Exercise 4: Load data 44
Exercise 5: Summary statistics 44
Exercise 6: Inline reporting 45
Exercise 7: Global eval 45
References 45

References **47**

Preface

This workshop is a hands-on introduction to writing reproducible, publication-ready research documents in R and Quarto. Across two sessions, we'll cover the full workflow from raw data to a formatted manuscript, including summary tables, model outputs, figures, citations, and export to HTML, PDF, Word and LaTeX.

Who this is for

Researchers with a working knowledge of R and the tidyverse who want to move from exploratory analysis to polished, reproducible manuscripts.

What you will learn

By the end of this workshop, you will be able to:

- Write and structure documents in Quarto using Markdown
- Export the same document to multiple formats (HTML, Word, PDF, LaTeX) with minimal format-specific tweaking
- Incorporate data directly into your text using inline reporting
- Produce publication-ready summary tables and model output tables
- Create and export figures with `ggplot2` and cross-reference them in your manuscript
- Manage code chunk options to control what appears in your output

Sessions

Day 1 focuses on the foundations: writing in Quarto, producing different output formats from a single source document, and bringing your data into the narrative through summary tables and inline reporting. This will be broken into the following chapters:

- Markdown basics: writing in Quarto
- Output formats: HTML, Word, PDF, and LaTeX
- Incorporating data: summary tables and inline reporting

Day 2 builds on this to cover the statistical reporting workflow: extracting and presenting model results, producing publication-ready figures, and tying everything together with cross-references. This will be broken into the following chapters:

- Reporting model results: model summary tables and inline reporting
- Producing figures: publication-ready plots, exporting, and cross-referencing

Tips on code chunk options are woven throughout both days.

How to use this book

Each chapter corresponds to roughly 30 minutes of workshop time. Code chunks are fully executable, work through them in your own RProject alongside the book. Each chapter also has accompanying slides (linked at the top) for the taught portion.

Required software

Before the first session, please make sure you have the following installed:

- **R** ($\geq$ 4.2): https://cran.r-project.org
- **RStudio** ($\geq$ 2022.07): https://posit.co/download/rstudio-desktop
- **Quarto** (latest): https://quarto.org/docs/download
- **TinyTeX** (for PDF output) — see Installing TinyTeX below

Installing R, RStudio, and Quarto

Download and install each from the links above, following the instructions for your operating system. Install them in order: R first, then RStudio, then Quarto.

Installing TinyTeX

TinyTeX is a lightweight LaTeX distribution installable directly from R:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

Checking your versions

In the R console:

```
R.version.string
tinytex::is_tinytex()
tinytex::tlmgr("--version")
```

In the **Terminal** tab:

```
quarto --version
```

RStudio's version is visible under *Help → About RStudio*.

Updating

- **R**: download the latest version from https://cran.r-project.org
- **RStudio**: *Help → Check for Updates*
- **Quarto**: download the latest version from https://quarto.org/docs/download
- **TinyTeX**: `tinytex::tlmgr_update()`

Packages

We'll be using the `pacman` package to load and install required (CRAN) packages. As long as you have this package installed you should be able to run the example and exercise code.

```
install.packages("pacman")
```

You only need to install `pacman` once. After that, `pacman::p_load()` handles loading and installing any missing packages automatically. Avoid leaving `install.packages()` calls uncommented in your scripts as they will re-run every time the script is executed or the document is rendered, which is slow and can cause unexpected package version changes. In other words, run this in the Console, or comment it out if you have it in a script.

Disclaimer

These materials were developed with the assistance of an AI language model (Claude, Anthropic) for outlining and editorial refinement using guided prompts. All content was fully reviewed, improved (I hope), and edited by the author. Every topic and reference included falls within the author's own expertise; the use of AI assistance was a practical decision about time and effort, not a substitution for subject knowledge. Any errors, inaccuracies, or omissions are the author's own responsibility.

These chapters also lean heavily on materials I previously created (without AI assistance) on R for Reproducibility, both in a semester-length course Open Science Practices: Implementing a Reproducible Analysis Workflow, a 4-day workshop Open Science Practices for Linguistic Research: Reproducible Analyses in R, and a 2-part workshop Reproducible Workflow in R. Unlike these previous materials which covered reproducibility and dynamic reports more broadly (and more in-depth), this book focuses on dynamic reports and reproducible analyses in the context of reporting results for publication and writing a manuscript in general.

Session 1: Quarto foundations

(R)Project Hygiene

RProjects, renv, and Folder Structure

Why project hygiene matters

Reproducibility begins before you write a single line of analysis. A clean, self-contained project structure ensures that collaborators, and your future self, can re-run everything without hunting for files or reinstalling packages.

The concept of *literate programming* (Knuth, 1984) established the principle that code and documentation should be written together, with human-readable explanation woven through the analysis rather than added as an afterthought. This philosophy underlies tools like R Markdown and Quarto, and extends naturally to project organisation: just as literate programming asks that code be written for human readers, good project hygiene asks that a project be structured for human navigators, i.e., people who need to understand, re-run, or build on your work without guidance from you directly.

Nagler (1995) outlines a set of principles for good computing practice in quantitative research that remain as relevant today as when they were written. Central among these are the ideas that code should be modular, well-documented, and organised so that the analysis can be reproduced from scratch by someone unfamiliar with the project. This means separating raw data from processed data, keeping analysis scripts independent and sequentially ordered, and never modifying raw data files directly. Bowers & Voors (2016) extend these principles to the context of modern social science, arguing that transparent and reproducible research practice is not merely a technical matter but a scientific norm, one that requires deliberate organisational habits from the outset of a project.

Good project organisation is also a matter of research data management (RDM). Funding bodies and journals increasingly require that data and analysis code be archived and made available alongside publications, in line with the FAIR principles (Wilkinson et al., 2016): (meta)data should be Findable, Accessible, Interoperable, and Reusable. A well-organised RProject is a practical first step towards FAIR-compliant research: clear folder structures, descriptive file names, and documented dependencies make it far easier to archive and share a project in a usable state. Conversely, a disorganised project that

only makes sense to its author is difficult to share meaningfully, regardless of whether the files are technically accessible.

Poor project organisation is one of the most common sources of reproducibility failures in practice. Files named `analysis_final_v2_ACTUAL.R`, data loaded with absolute paths like `C:/Users/myname/Desktop/study1/data.csv`, and package dependencies that are never documented all make it difficult or impossible for others (or your future self after six months away from the project) to reproduce your work.

The habits covered in this chapter, namely consistent folder structure, relative file paths via `here::here()`, and package management with `renv`, are low-cost investments that pay dividends throughout the life of a project and beyond.

RProjects

Always work inside an RProject (`.Rproj`). This sets the working directory to the project root, makes `here::here()` reliable, and keeps your R environment isolated from other projects. Without an RProject, file paths are fragile and environment state can bleed between sessions.

An RProject is simply a folder with a `.Rproj` file at its root. Opening this file in RStudio or Positron sets the working directory automatically — no `setwd()` required.

Creating a new RProject

In RStudio: *File → New Project → New Directory → New Project*. Tick "Use renv with this project" to initialise package management from the start.

In Positron: *File → New Folder → initialise as R project*.

> **Tip**
>
> Get into the habit of always opening your work via the `.Rproj` file rather than navigating to individual scripts. This ensures the working directory is always set correctly and that `here::here()` resolves paths from the project root.

Folder structure

A consistent folder structure makes projects navigable by collaborators and reviewers, and is a prerequisite for FAIR-compliant data sharing (Wilkinson et al., 2016). The exact structure will vary by project, but the following is a reasonable starting point for a research project with data, analysis scripts, and a manuscript:

```
my-project/
|-- my-project.Rproj
|-- _quarto.yml              # if using Quarto
|-- renv.lock                # package versions
|-- README.md                # project overview
|-- data/
|   |-- raw/                 # original data, never modified
|   `-- processed/           # cleaned and anonymised outputs
|-- R/
|   |-- 01-anonymise.R
|   |-- 02-process.R
|   |-- 03-eda.R
|   `-- 04-analysis.R
|-- manuscript/
`-- output/
    |-- figures/
    |-- tables/
    `-- models/
```

The `R/` folder contains numbered scripts following a modular analysis workflow (Nagler, 1995): each script has a single responsibility, takes clearly defined inputs, and produces clearly defined outputs. Scripts are numbered to indicate the order in which they should be run. The `data/raw/` folder contains the original, unmodified data — this is the source of truth and should never be altered directly.

⚠ Data protection: raw and identifiable data

If your project involves personal, sensitive, or identifiable data (e.g. recordings, transcripts, demographic information), keep the following in mind:

- **Never commit raw data to version control.** Add `data/raw/` to your `.gitignore` before your first commit. This prevents raw data from ever being tracked accidentally.
- **Never share the project folder** (e.g. as a zip, via GitHub, or cloud sync) without first checking what `data/raw/` contains.
- **Pseudonymisation and anonymisation come first.** Only once raw data has been processed and (pseudo)anonymised should derived outputs be treated as shareable.
- **Archive, then remove.** Ideally, raw data lives in `data/raw/` only during active processing. Once archived to a secure location, remove it from the project folder.

The `data/processed/` folder should contain only anonymised or non-identifiable outputs.

Project-relative file paths

Absolute file paths like `C:/Users/myname/Desktop/study1/data.csv` break the moment the project is moved or shared. Use `here::here()` for all file paths so the project is portable across machines and operating systems:

```
# load raw data
df_raw <- readr::read_csv(here::here("data", "raw", "mydata.csv"))

# save processed data
saveRDS(df_clean, here::here("data", "processed", "data_clean.rds"))

# load a saved model
m1 <- readRDS(here::here("output", "models", "m1.rds"))
```

`here::here()` builds the file path from the project root — the folder containing the `.Rproj` file, regardless of where the script is located within the project. This means a script in `R/04-analysis.R` and a `.qmd` file in `manuscript/` can both use `here::here("data", "processed", "data_clean.rds")` and resolve to the same file.

> **Tip**
>
> Never use `setwd()` in a script that will be shared or re-run. It hardcodes a path that only works on your machine. `here::here()` is always the better alternative inside an RProject.

renv: reproducible package management

`renv` records the exact version of every package your project uses in a lockfile (`renv.lock`). Anyone who clones your project and runs `renv::restore()` gets an identical package environment — no more "works on my machine" failures due to package version differences.

```
renv::init()        # initialise once at project start
renv::snapshot()    # update lockfile after installing new packages
renv::restore()     # recreate environment from lockfile
```

The `renv.lock` file should be committed to version control. The `renv/library/` folder should not — it is machine-specific and can be large.

> **Warning**
>
> `renv` solves package version differences between collaborators but has limits over long time horizons. R itself, system libraries, and compiled packages can change in ways that `renv` cannot fully protect against. Document your R version, operating system, and any system dependencies in your `README.md` for long-term reproducibility.

An alternative: pacman

If you prefer not to use `renv`, `pacman::p_load()` provides a lightweight alternative that installs missing packages automatically on load:

```
install.packages("pacman")  # run once
pacman::p_load(tidyverse, here, broom, gt, kableExtra)
```

`pacman::p_load()` checks whether each package is installed, installs any that are missing, and loads all of them. This does not lock package versions, so it is less reproducible than `renv` over time, but it is simpler and sufficient for following along in a workshop setting.

> **Warning**
>
> Avoid leaving `install.packages()` calls uncommented in scripts. They will re-run every time the script is executed or the document is rendered, which is slow and can cause unexpected package version changes.

Quarto in an RProject

If your project produces a Quarto book or report, place `_quarto.yml` at the project root. Quarto and RStudio/Positron detect it automatically and use it to configure rendering for the entire project.

For a standalone manuscript or report (not a book), place the `.qmd` file at the project root alongside the `.Rproj` file. The YAML header of the `.qmd` file controls rendering in this case.

> **Tip**
>
> A Quarto project and an RProject are complementary but independent. The `.Rproj` file manages the R environment and working directory; the `_quarto.yml` file manages document rendering. Both can coexist in the same folder, and together

> they provide a fully self-contained, reproducible research project.

Chapter slides

Writing in Quarto

Markdown, Citations, and Manuscript Formatting

Purpose

This chapter covers writing in Quarto independently of any data or analysis. This is relevant if you want to write any part of your manuscript (e,g., the introduction, methods, or results section) directly in Quarto, rather than drafting in Word or Overleaf and manually inserting statistics afterwards.

Quarto is an implementation of *literate programming* (Knuth, 1984), whereby code and prose are written together in a single document. The document is then rendered to a readable output format, in this case HTML, Word, or PDF. This approach has been popularised in the R community through R Markdown (Xie et al., 2018, 2020), of which Quarto (Allaire et al., 2024) is the next-generation successor, with broader language support and improved output options.

Writing in Quarto means that your prose and your analysis live in the same document. Values reported in the text can be pulled directly from your data and models using inline R code, so they update automatically when analyses change. This eliminates a common source of error in academic writing: the manual transcription of numbers from statistical software into a word processor. For example, Nuijten et al. (2016) found that around 50% of psychology papers contained at least one statistical reporting error, many attributable to exactly this kind of manual transcription.

This chapter focuses on the writing side: Markdown syntax, document structure, citations, and cross-references. Connecting prose to data and models is covered in later chapters.

Text formatting

Quarto uses Markdown for text formatting. Markdown is a lightweight plain-text syntax that is readable in its raw form and renders to formatted output. Unlike Word or Overleaf,

formatting in Quarto is entirely handled by the source document: there are no toolbar buttons or style menus, only text.

The most common formatting options are:

Markdown	Output
`**bold**`	**bold**
`*italic*`	*italic*
`***bold italic***`	***bold italic***
`` `code` ``	`code`
`~~strikethrough~~`	~~strikethrough~~
`H~2~O`	H_2O
`R^2^`	R^2

Markdown formatting is intentionally minimal. For most academic writing purposes (prose, headings, lists, tables, and equations) it covers everything needed without requiring knowledge of LaTeX or HTML. That being said, I still highly recommend having some LaTeX familiarity, as it's commonly used by linguists and might be helpful with future collaborations.

APA 7 statistical symbols

APA 7 requires statistical symbols to be italicised in text. In Markdown, wrapping text in `*...*` produces italics, which renders correctly in HTML, Word, and PDF output:

Statistic	Markdown	Output
Mean	`*M* = 310.8`	$M = 310.8$
SD	`*SD* = 155.9`	$SD = 155.9$
t-test	`*t*(48) = 2.31`	$t(48) = 2.31$
p-value	`*p* < .001`	$p < .001$

> **Tip**
>
> APA 7 omits the zero before the decimal point for values that cannot exceed 1 (e.g. $p = 0.031$ is incorrect; $p = .031$ is correct). This applies to correlation coefficients, proportions, and p-values. Inline R code can be written to apply this formatting automatically — see Section for an example.

Headers

Use `##` for first-level sections within a chapter, `###` for subsections. The chapter title is set in the YAML `title:` field:

```
## Methods
### Participants
#### Inclusion criteria
```

> **Tip**
>
> In a Quarto book, avoid using `#` (H1) inside chapter files — it is reserved for the chapter title set in the YAML.

Lists and callouts

```
- unordered item
  - nested item

1. ordered item
   1. nested item
```

Callout boxes draw attention to key points, and can include a title if you specify one:

```
::: callout-note
## This tip has a title
A tip.
:::
```

> **Tip: adding a title**
>
> A tip.

```
::: callout-note
A note.
:::
```

> **Note**
>
> A note.

```
::: callout-warning
A warning.
:::
```

> **Warning**
>
> A warning.

```
::: {.callout-important collapse="true"}
## Important: collapsing a callout
An important point. You can also collapse a callout box. This only works in HTML ou
:::
```

> **Important: collapsing a callout**
>
> An important point. You can also collapse a callout box. This only works in HTML output.

Equations

Write LaTeX math inline with `$...$` or in a display block with `$$...$$`:

```
The model was specified as:

$$
y_i = \beta_0 + \beta_1 \text{Tense}_i + \beta_2 \text{Lifetime}_i + \varepsilon_i
$$ {#eq-model}
```

This is then rendered as:

$$y_i = \beta_0 + \beta_1 \text{Tense}_i + \beta_2 \text{Lifetime}_i + \varepsilon_i \tag{0.1}$$

Reference in prose with `@eq-model` → "Equation 0.1"

> **Tip**
>
> If you need the LaTeX code for a mathematical symbol or expression, hover over any rendered equation in an HTML document and right-click. Select **Math Settings → TeX Commands** to display the underlying LaTeX source. This is useful for replicating equation formatting or looking up the correct LaTeX syntax for a symbol.

Cross-references

Cross-references allow you to refer to figures, tables, sections, and equations by label rather than by number. Quarto resolves the labels to the correct numbers at render time, so adding or reordering elements never requires manually updating references in the text.

Every cross-reference label must begin with a type-specific prefix. The prefix tells Quarto what kind of element is being referenced and how to format the reference in the output:

Type	Prefix	Reference syntax	Example output
Figure	`fig-`	`@fig-rt`	Figure 1
Table	`tbl-`	`@tbl-desc`	Table 1
Section	`sec-`	`@sec-results`	Section 3
Equation	`eq-`	`@eq-model`	Equation 1

Labels are set in the chunk options for figures and tables, and in the heading syntax for sections. A label without the correct prefix will not be recognised as a cross-reference and will render as plain text.

> **Figures and Tables**
>
> We'll cover tables later today, and figures in the next session.

Sections

Add a label to any heading using the `{#sec-}` syntax:

```
## Results {#sec-results}
## Discussion {#sec-discussion}
```

Reference in prose:

```
The implications of these findings are discussed in @sec-discussion.
```

> **Tip**
>
> Cross-references are one of the most practical advantages of writing in Quarto (or LaTeX) over Word. In Word, renumbering figures after adding one to the middle of a document requires updating every reference manually. In Quarto, the numbers are always correct automatically.

Numbered example sentences

Pandoc's `(@)` syntax auto-numbers linguistic examples continuously across the document:

```
(@cond-a) Example sentence 1.
(@cond-b) Example sentence 2.

Sentences (@cond-a) and (@cond-b)...
```

(1) Example sentence 1.
(2) Example sentence 2.

Sentences (1) and (2)...

> **Tip**
>
> For sub-examples (1a, 1b), interlinear glosses, or full linguistic formatting in PDF output, use the `linguex` or `gb4e` LaTeX packages via `include-in-header` in your YAML.

Citations

Quarto handles citations through Pandoc's citation processing system. References are stored in a BibTeX file (`.bib`) and formatted according to a Citation Style Language (`.csl`) file. This means citation formatting is automatic and consistent, and switching between citation styles (e.g. APA to Chicago) requires only changing the `.csl` file.

Add the following to your document YAML:

```yaml
bibliography: references.bib
csl: apa.csl
```

The `.bib` file contains structured reference entries in BibTeX format. The `.csl` file controls how those references are formatted in the output. Both files should be placed in your project root, or referenced with a relative path.

Download `apa.csl` from the Zotero style repository:

```r
download.file(
  "https://www.zotero.org/styles/apa",
  here::here("apa.csl")
)
```

Citation syntax

Citations are inserted using `@key` syntax, where `key` is the identifier from your `.bib` entry:

Syntax	Output
`@Bates2015`	Bates et al. (2015)
`[@Bates2015]`	(Bates et al., 2015)
`[-@Bates2015]`	(2015)
`[@Bates2015, p. 12]`	(Bates et al., 2015, p. 12)
`[@Bates2015; @R2024]`	(Bates et al., 2015; R Core Team, 2024)

These can be added in-text, so `As described in @Bates2015...` would be rendered as: *As described in Bates et al. (2015)....*

Getting BibTeX entries

BibTeX entries can be obtained from several sources:

- **Zotero** — export individual items or your full library as BibTeX. This is the recommended workflow for managing references across a project.
- **Google Scholar** — click Cite below a result, then select BibTeX to copy the entry.
- **DOI** — paste a DOI at `doi2bib.org` to generate a BibTeX entry automatically.
- **R packages** — `toBibtex(citation("lme4"))` generates a citable BibTeX entry for any R package. Always cite the packages you use.

References section

Add a references section at the end of your document. Quarto will populate it automatically with all cited works:

```
## References {.unnumbered}

::: {#refs}
:::
```

The `.unnumbered` class prevents the References heading from receiving a section number, which is standard in APA manuscripts.

Inline reporting

Inline R code allows values computed in R to be embedded directly in prose. Rather than typing a number manually, you write a small R expression that is evaluated at render time and replaced with its result. This means reported values are always consistent with the underlying analysis, and update automatically if the data or model changes.

The syntax for inline R code is a backtick followed by `r`, a space, the R expression, and a closing backtick. For example:

```
The data were collected from ` r n_distinct(df$px)` participants. The mean total re
```

This is particularly useful for results sections, where small changes to preprocessing or exclusion criteria can affect many reported values. With inline reporting, re-running the analysis and re-rendering the document is sufficient to update all reported statistics.

Rounding and formatting

Use `round()` to control decimal places. For p-values, a small helper avoids reporting exact values below a threshold:

```r
fmt_p <- function(p) {
  ifelse(p < .001,
  "< .001",
  paste0("= ", round(p, 3)))
}
```

This function (`fmt_p()`) will print p-values smaller than .001 as $p < .001$, printing all other p-values to three decimal points. You can also create a function to print $< .01$ and $< .05$ when applicable (I would do this using the `case_when()` function). This can then be used inline (but note that you should also report the estimate, standard error, and t- or z-values):

```
The effect was significant (*p* ` r fmt_p(p_value)`).
```

Connecting to models

You can also extract estimates directly from fitted model objects and report them inline, rather than typing them out manually. This makes your results robust to upstream changes: if you refit a model, the reported values update automatically. This is covered in depth in the model reporting chapter.

> **Tip**
>
> Any number that appears in your manuscript should be computed, not typed. This is the single most effective way to eliminate reporting errors in academic writing.

References

Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., & Dervieux, C. (2024). *Quarto*. https://github.com/quarto-dev/quarto-cli

Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, *67*(1), 1–48. https://doi.org/10.18637/jss.v067.i01

Bowers, J., & Voors, M. (2016). How to improve your relationship with your future self. *Revista de Ciencia Política*, *36*(3), 829–848. https://doi.org/10.4067/S0718-090X2016000300011

Knuth, D. E. (1984). Literate programming. *The Computer Journal*, *27*(2), 97–111. https://doi.org/10.1093/comjnl/27.2.97

Nagler, J. (1995). Coding style and good computing practices. *PS: Political Science and Politics*, *28*(3), 488–492. https://doi.org/10.2307/420315

Nuijten, M. B., Hartgerink, C. H. J., Assen, M. A. L. M. van, Epskamp, S., & Wicherts, J. M. (2016). The prevalence of statistical reporting errors in psychology (1985–2013). *Behavior Research Methods*, *48*, 1205–1226. https://doi.org/10.3758/s13428-015-0664-2

Palleschi, D., Ronderos, C. R., & Knoeferle, P. (2025). Living in the present – how referent lifetime influences processing of past, present (perfect), and future tenses. *Glossa Psycholinguistics*, *4*(1), 1–48. https://doi.org/https://doi.org/10.5070/G601119481

Wilkinson, M. D. et al. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, *3*, 160018. https://doi.org/10.1038/sdata.2016.18

Xie, Y., Allaire, J. J., & Grolemund, G. (2018). *R markdown: The definitive guide*. Chapman; Hall/CRC. https://bookdown.org/yihui/rmarkdown

Xie, Y., Dervieux, C., & Riederer, E. (2020). *R markdown cookbook*. Chapman; Hall/CRC. https://bookdown.org/yihui/rmarkdown-cookbook

Output Formats

HTML, Word, PDF, and LaTeX

Purpose

This chapter covers rendering a Quarto document to different output formats. The same `.qmd` source file can produce HTML, Word, PDF, and a LaTeX `.tex` file, but each format has specific YAML options and some practical differences worth understanding before committing to a workflow.

Rather than trying to support all formats simultaneously, the most pragmatic approach is to decide early which format best fits your collaboration context. The right choice depends on who your co-authors are, how they prefer to give feedback, and what the target journal requires. A workflow that is technically reproducible but creates friction for collaborators is unlikely to be sustainable in practice.

To illustrate: my own workflow has evolved considerably over time. Writing my dissertation entirely in R Markdown (with separate `.Rmd` files per chapter knitted together) worked well for a sole-author project but would have been impractical with collaborators. Sharing PDF output for feedback from co-authors on a journal article was workable but painful: comments arrived as annotations on a PDF, had to be manually transferred back to the source (by me), and any revision meant re-rendering and re-sharing (2/10, would not recommend). More recently, rendering a first draft from Rmarkdown to PDF, uploading the produced `.tex` document to Overleaf, and inviting collaborators to edit there directly has been a reasonable compromise, as it preserves the reproducible analysis pipeline while giving collaborators a familiar LaTeX editing environment (8/10: not perfect for reproducibility at the writing stage, but it facilitates collaboration in a way that a pure Quarto workflow currently cannot).

A further option is to collaborate in Google Docs, rendering results text and tables to Word and copying them across. This is admittedly imperfect from a reproducibility standpoint, as prose edits made in Google Docs are not tracked in the source `.qmd`. However, it can be a pragmatic workflow when collaborators are not willing or able to use anything else. Crucially, the analysis itself remains fully documented in R scripts and

output files, even if the manuscript prose lives elsewhere. The results are still reproducible; only the writing layer is decoupled.

The sections below cover the main output formats, their YAML options, and the practical trade-offs involved.

Rendering formats

Quarto separates the source document from its output format. The same `.qmd` file can be rendered to multiple formats by specifying them in the YAML `format:` block:

```yaml
format:
  html:
    embed-resources: true
  pdf:
    keep-tex: true
  docx:
    reference-doc: custom-reference.docx
```

Render all formats at once from the terminal:

```
quarto render manuscript.qmd
```

Or a single format:

```
quarto render manuscript.qmd --to docx
```

This separation of content from presentation is one of Quarto's core strengths: you write once and render to whatever format a collaborator, journal, or audience requires, without maintaining separate documents. However, I suggest you choose *one* output format until you have a specific need to render to different formats.

HTML

HTML is the most flexible output format. It supports interactive elements, renders equations via MathJax, and can be hosted online (like this book, written in Quarto) or shared as a standalone file. For internal reports, supplementary materials, or workshop books, HTML is usually the best default.

```yaml
format:
  html:
    toc: true
    toc-depth: 3
    number-sections: true
    embed-resources: true
    theme: flatly
```

> **Self-contained HTML with `embed-resources`**
>
> By default, Quarto renders HTML with external dependencies (CSS, JavaScript, figures) stored in a separate folder alongside the `.html` file. This means sharing the file alone — by email, for example — will result in missing images and broken styling. Setting `embed-resources: true` bundles everything into a single self-contained `.html` file that can be shared without any accompanying folder.
> The trade-off is file size and render time: `embed-resources: true` produces larger files and renders more slowly. For a book or website that will be hosted online and served from a web server, leave it as `false`.

Word

Word output is the most practical format for sharing with co-authors who are not using Quarto, and is widely accepted by journals for submission. The rendered `.docx` looks and behaves like any other Word document, co-authors can annotate, track changes, and comment without needing to know anything about Quarto.

```yaml
format:
  docx:
    reference-doc: custom-reference.docx
    toc: false
    number-sections: false
```

A reference `.docx` controls the visual styling of the output (fonts, margins, heading styles, and spacing) by mapping Quarto's document elements to Word styles. Generate a default one and then customise it:

```bash
quarto pandoc -o custom-reference.docx --print-default-data-file reference.docx
```

Open the generated file in Word, modify the paragraph and heading styles to match your journal's requirements, and point your YAML to it with `reference-doc: custom-reference.docx`.

> **Citations in Word**
>
> Citations are rendered as plain formatted text in `.docx` output. They are not live fields linked to your `.bib` file, so editing them directly in Word — or adding new ones — will have no effect on the source document and will be overwritten on re-render. Always treat the `.qmd` as the source of truth, and re-render to update the `.docx` rather than editing it directly.
> If co-authors need to edit citations in Word, the most practical solution is to use Zotero's Word plugin for the final submission copy, keeping the Quarto document as the reproducible record of the analysis.

PDF

PDF output is produced by rendering the `.qmd` to LaTeX and then compiling with a LaTeX engine (by default `pdflatex`). This gives access to the full power of LaTeX for typesetting, including precise control over layout, fonts, and mathematical notation. PDF is the appropriate format for journal submission when a `.pdf` or `.tex` file is required.

```yaml
format:
  pdf:
    documentclass: article   # or scrartcl, apa7
    papersize: a4
    fontsize: 12pt
    geometry: margin=2.5cm
    linestretch: 1.5
    number-sections: true
    keep-tex: true
    include-in-header:
      text: |
        \usepackage{booktabs}    % professional table rules
        \usepackage{longtable}   % tables spanning multiple pages
        \usepackage{microtype}   % improved text justification
```

> **keep-tex**
>
> Setting `keep-tex: true` saves the intermediate `.tex` file alongside the rendered PDF. This is useful in two situations: if the journal requires a raw `.tex` file for submission, or if you want to upload to Overleaf for final formatting tweaks or co-author editing. See Section for important caveats about citation handling in `.tex` output.

Table 1: Comparison of output formats for common manuscript features.

Feature	HTML	PDF	Word
Citations	yes from .bib	yes from .bib	yes rendered as text
Live .bib link	yes	limited depends on cite-method	no
Cross-references	yes	yes	yes
Equations	yes MathJax	yes native LaTeX	limited limited
Table formatting	‘gt‘	‘kableExtra‘	‘flextable‘
Custom styles	CSS	LaTeX	reference .docx
Self-contained	‘embed-resources: true‘	always	always

Format comparison

The table below summarises the key differences between output formats for common manuscript features.

APA 7 and `.tex` output

The following considerations are only relevant if you intend to edit the `.tex` file after rendering — for example, uploading to Overleaf for final formatting or co-author editing. If you render directly to PDF, Word, or HTML from Quarto, citations will be formatted correctly by the default citeproc engine regardless of which option you choose. The issue is specifically about whether the `.tex` file contains live `\cite{}` commands that a LaTeX editor like Overleaf can resolve, or plain formatted text that it cannot.

For `.tex` editing workflows, there is no perfect solution. The three main options involve trade-offs between APA 7 accuracy and Overleaf compatibility:

citeproc (default)
Uses `csl: apa.csl` to format citations at render time. Produces the most accurate APA 7 output of the three options, but resolves citations to plain text in the `.tex` file. Overleaf receives a fully formatted document but with no live citation commands — adding or editing references requires going back to the Quarto source.

natbib + apacite
Passes citations through to LaTeX as live `\cite{}` commands, which Overleaf can resolve using your uploaded `.bib` file. However, `apacite` was written to APA 6th edition standards and has known formatting issues with APA 7 requirements, including author name truncation rules and DOI formatting.

Table 2: Citation backend options for .tex output.

Option	APA 7 accuracy	Live .tex citations	Overleaf
citeproc + apa.csl	yes best	no	limited upload .tex only
natbib + apacite	limited APA 6	yes	yes with .bib
biblatex + biblatex-apa	yes good	yes	yes set biber in compiler

biblatex + biblatex-apa
The most APA 7-compliant LaTeX citation option, with live citations. Requires `biber` as the backend processor, which must be set manually in Overleaf's compiler settings. More complex to set up than the other options but the best choice if both APA 7 accuracy and live Overleaf citations are required.

Choose based on your priority: APA 7 accuracy or Overleaf compatibility. To switch from the default citeproc:

```
# natbib
format:
  pdf:
    keep-tex: true
    cite-method: natbib
    include-in-header:
      text: |
        \usepackage[natbibapa]{apacite}

# biblatex
format:
  pdf:
    keep-tex: true
    cite-method: biblatex
    biblio-style: apa
```

Chapter slides

Data and Tables

Code Chunks, Loading Data, and Summary Tables

Purpose

This chapter introduces the practical side of working with data in Quarto. Rather than running analyses in a separate R script and copying results into a document, Quarto allows you to load data, compute summaries, and produce formatted tables directly within the manuscript. The result is a document in which every number, table, and figure is generated from the underlying data at render time.

We cover the essentials of code chunk options, loading data from file, producing summary tables with `gt` and `kableExtra`, cross-referencing tables in prose, and reporting values inline. Together these form the foundation of a reproducible results section.

Code chunks

Code chunks are the blocks of R code embedded in a `.qmd` document. They are delimited by triple backticks and a language identifier:

```
```{r}
rt <- c(312, 287, 445, 301, 389)
mean(rt)
```
```

```
#> [1] 346.8
```

Each chunk is executed in sequence when the document is rendered, and the output (whether a number, a table, or a figure) is inserted into the document at that position.

Table 1: Common code chunk options for manuscript writing.

Option	Values	Effect
echo	true / false	Show or hide the code
output	true / false	Show or hide the output
eval	true / false	Run or skip the chunk
include	true / false	Include chunk and output in document
warning	true / false	Show or hide warnings
message	true / false	Show or hide messages
label	string	Chunk identifier for cross-references
fig-cap	string	Figure caption
tab-cap	string	Table caption

Note: Quarto beyond R

While this workshop focuses on R, Quarto supports multiple languages in code chunks, including Python (`{python}`), Julia (`{julia}`), and Observable JavaScript (`{ojs}`). Different languages can be used in the same document without any additional packages, with each chunk executed by its respective engine. You can use the `reticulate` package if you want to share objects between R and Python chunks within the same session (and would need it to render Python chunks to PDF).
For example, a Python chunk in the same document as R code:

```{python}
import numpy as np # numpy is Python's main numerical computing library, similar t

rt = [312, 287, 445, 301, 389]
print(np.mean(rt))
```

```
#> 346.8
```

Chunk options

Chunk behaviour is controlled by options written at the top of the chunk using the `#|` prefix. The most important options for manuscript writing are:

For a manuscript, you typically want to hide code and show only output. Set global defaults in `_quarto.yml` or the document YAML, then override per chunk where needed:

```yaml
execute:
  echo: false      # hide code from output
  warning: false   # suppress R warnings
  message: false   # suppress R messages (e.g. package loading messages)
```

Then override for specific chunks:

```
```{r}
#| echo: true
#| label: tbl-desc
#| tbl-cap: "Descriptive statistics by condition."
tbl_demo |> gt()
```
```

> **`echo: true` or `false`?**
>
> The right default depends on your document's purpose. For teaching materials and workshop scripts, `echo: true` ensures participants can see the code alongside the output. For manuscript submissions, `echo: false` (as well as `warning: false` and `message: false`) produces clean output with no visible code. For internal analysis reports, i.e., documents whose purpose is to serve as a transparent record of your analysis, `echo: true` is essential for auditability and reproducibility.

Chunk labels

Every chunk that produces a figure or table should have a label. Labels must begin with the correct prefix (`fig-` for figures, `tbl-` for tables) and be unique across the document. Unlabelled chunks cannot be cross-referenced.

```
#| label: tbl-desc
#| tbl-cap: "Descriptive statistics by condition."
```

> **Warning**
>
> Duplicate labels cause render errors. If you copy a chunk, always update the label.

Loading data

Data should be stored in `data/raw/` and loaded at the top of the document using `here::here()`. This ensures the file path works regardless of where the project is opened:

```
df <- readr::read_csv(here::here("data", "raw", "your-data.csv"))
```

For processed data or saved R objects, load from `data/processed/` or `output/models/`:

```
df_clean <- readRDS(here::here("data", "processed", "data_example.rds"))
lmer_verb_firstfix <- readRDS(here::here("output", "models", "lmer_verb_firstfix.rd
```

> **Tip**
>
> Load all data objects in a single setup chunk at the top of the document with `#| output: false`. This keeps the document clean and makes dependencies explicit.

Saving processed data

After cleaning and transforming your data, save the processed object so it can be loaded quickly in subsequent documents without re-running the full pipeline:

```
saveRDS(df_clean, here::here("data", "processed", "data_example.rds"))
```

Inline reporting

Values from your data can be embedded directly in prose using inline R code. This ensures reported statistics are always consistent with the underlying data. For frequently reported values, compute them once in a code chunk and store as named objects, then reference those objects inline:

```
n_px  <- dplyr::n_distinct(df_clean$px)
n_obs <- nrow(df_clean |> filter(region == "verb"))
m_tt  <- round(mean(df_clean$tt, na.rm = TRUE), 1)
sd_tt <- round(sd(df_clean$tt,   na.rm = TRUE), 1)
```

```
The sample comprised ` r n_px` participants (*N* observations = ` r n_obs`).
Mean total reading time was ` r m_tt` ms (*SD* = ` r sd_tt`).
```

> **Tip**
>
> Storing computed values as named objects keeps inline expressions short and readable. It also means the value is only computed once, reducing the risk of inconsistencies if the expression is used in multiple places.

Chapter slides

Session 2: Model Outputs & Reporting

Exercises

Set-up

Before you begin

These exercises require you to be working in Quarto within an RProject, ideally the same RProject designed for this workshop (https://github.com/daniela-palleschi/publishr-workshop). Choose one of the two setup options below depending on your situation.

Option A: Use the workshop repository (recommended)

With Git

This requires Git to be installed. Check by running `git --version` in the terminal — if it is not installed, download it from git-scm.com.

Before cloning, decide whether to use HTTPS or SSH. If you are unsure, use HTTPS. SSH requires keys to be configured with GitHub but avoids needing to enter credentials each time.

- **HTTPS URL:** `https://github.com/daniela-palleschi/publishr-workshop.git`
- **SSH URL:** `git@github.com:daniela-palleschi/publishr-workshop.git`

Via the terminal:

```
git clone https://github.com/daniela-palleschi/publishr-workshop.git
```

Via RStudio: *File → New Project → Version Control → Git*, then paste the URL into the dialog box.

Via Positron: *File → New Folder from Git Repository*, then paste the URL.

Without Git

1. Go to github.com/daniela-palleschi/publishr-workshop
2. Click the green **Code** button and select **Download ZIP**
3. Unzip the folder somewhere sensible on your machine

After cloning or downloading

1. Open `publishr-workshop.Rproj` in RStudio or Positron
2. Copy your data file to `data/raw/` (or use the sample dataset; see below)
3. Open `exercises/day1.qmd` and work through the exercises

Option B: Set up your own RProject

If you would prefer to use your own project:

1. Create a new RProject: *File → New Project → New Directory → New Project*
2. Create the following folder structure in the terminal:

```
mkdir -p data/raw data/processed exercises output/figures output/tables output/mode
```

3. Download the sample dataset (a subset from Palleschi et al. (2025); https://osf.io/6sra7) and save it to `data/raw/`:

Download the example dataset at: https://raw.githubusercontent.com/daniela-palleschi/publishr-workshop/main/data/raw/data_example.csv

4. Create a new Quarto document: *File → New File → Quarto Document*, and save it as `exercises/day1.qmd`
5. Install the required packages:

```
install.packages("pacman")
pacman::p_load(here, tidyverse, gt, kableExtra, broom, modelsummary)
```

> **Sample dataset**
>
> The sample dataset used in these exercises is a subset of a reading time study with the following structure: one row per word region per trial per participant, with columns including `px` (participant ID), `item` (item ID), `region` (sentence region), `tense`, `lifetime`, `gaze` (first-pass reading time), `tt` (total reading time), `rpd` (regression path duration), `first_fix` (first fixation duration), and `rating` (plausibility rating, 1–7).

> **Tip**
>
> If you are using Option B and your data has different column names, adapt the code in the exercises accordingly. The key variables used are `region`, `gaze`, `tt`, and `rating`.

Session 1 Exercises

In the `publishr_workshop` RProject, open the `exercises/session1.qmd` Quarto script.

Writing in Quarto

Exercise 1: Text formatting

In the script you'll see an example sentence (repeated below). Add some text formatting to it: try making some words bold, some italic, and add at least one superscript or subscript.

> The mean first-pass reading time at the verb region was 312 ms (SD = 159 ms).

Exercise 2: Lists and callouts

Add an unordered list of three things you want to get out of this workshop. Then add a callout box (any type) with a note about your dataset.

Exercise 3: Headers

Add the following headers to structure this document:

- A level-2 header called "Methods"
- A level-3 header called "Participants"
- A level-3 header called "Materials"

Exercise 4: Cross-references

Add the label `{#sec-methods}` to your Methods header from Exercise 3. Then write a sentence below that cross-references it using `@sec-methods`.

Exercise 5: Citations

Add a citation to your methods section. You will need to:

1. Add `bibliography: references.bib` to your YAML (already done above)
2. Add `csl: apa.csl` to your YAML (already done above)
3. Add at least one entry to `references.bib` — you can use the R citation:

```
toBibtex(citation("base"))
```

4. Cite it in prose using `@key` or `[@key]` syntax

Exercise 6: References section

Add a references section at the end of this document so your citations are listed. Remember to use the `.unnumbered` class on the heading.

Exercise 7: Inline arithmetic

Use inline R code to compute and report the following directly in prose:

Complete the sentence below:

2 + 2 = ____, and the square root of 144 is ____.

Output formats

Add appropriate headers below to keep your work structured.

Exercise 1: HTML with TOC

The YAML above already has a `toc: true` option under `html`. Render this document to HTML and check that the table of contents appears. Try changing `toc-depth` to 2 and re-render — what changes?

Exercise 2: Word output

Change the active output format to `docx` and render. Open the `.docx` file and inspect your citation. How is it formatted? Can you edit it directly in Word?

> **Note**
>
> To render to a specific format from the terminal: `quarto render exercises/day1.qmd --to docx`

Exercise 3: PDF and `.tex` output (stretch)

The YAML already has `keep-tex: true` under `pdf`. Render to PDF:

```
quarto render exercises/day1.qmd --to pdf
```

Then:

1. Open the PDF — does your citation look correct?
2. Open the `.tex` file — is your citation a live `\cite{}` command or plain text?
3. If you want to experiment with live citations in the `.tex` file, try adding `cite-method: natbib` to your PDF YAML options and re-render. Does the citation look different in the PDF? In the `.tex`?

> **This is a stretch exercise**
>
> Getting APA 7 formatting with live `.tex` citations working perfectly involves trade-offs — see the Output Formats chapter for a full discussion. The goal here is simply to observe the difference, not necessarily to resolve it.

Data and tables

Add appropriate headers below to keep your work structured.

Exercise 1: A simple code chunk

Add a code chunk below that computes `2 + 2` and renders it. Make sure the chunk is both printed (code visible) and evaluated (output visible).

Exercise 2: Echo without eval

Change the chunk options on your Exercise 1 chunk so that the code is printed but not evaluated. Render and observe the difference.

> **Tip**
>
> Relevant chunk option: `eval: false`

Exercise 3: Silent chunk

Add a new code chunk that computes `10 * 10`. Set the chunk options so that neither the code nor the output appears in the rendered document. Render and verify it is invisible.

> **Tip**
>
> Relevant chunk option: `include: false`

Exercise 4: Load data

Add a code chunk that loads the sample dataset. Set the chunk options so that the code is not printed in the rendered output but the chunk is evaluated.

```
df <- readr::read_csv(here::here("data", "raw", "your-data.csv"))
```

> **Tip**
>
> Relevant chunk option: `echo: false`

Exercise 5: Summary statistics

Create a code chunk that filters to the verb region and computes the mean and SD of first-pass reading time (`gaze`), storing them as named objects. Set the chunk so it runs silently.

```
m_gaze  <- # YOUR CODE HERE
sd_gaze <- # YOUR CODE HERE
```

> **Tip**
>
> You will need `filter(region == "verb")`, `mean(..., na.rm = TRUE)`, `sd(..., na.rm = TRUE)`, and `round(..., 1)`.

Exercise 6: Inline reporting

Report your values from Exercise 5 inline by completing the sentence below:

Mean first-pass reading time at the verb region was _____ ms (SD = _____ ms).

Exercise 7: Global eval

Add the following to your YAML `execute:` block:

```
execute:
  eval: false
```

Render. What happens? Which exercises break and why?

Now fix the problem by adding `#| eval: true` to the chunks that need to run (Exercises 4 and 5). Re-render and confirm everything works.

> **Note**
>
> This exercise illustrates why global chunk options need to be set carefully, and why it is often better to set `eval: false` only on specific chunks rather than globally.

References

Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., & Dervieux, C. (2024). *Quarto*. https://github.com/quarto-dev/quarto-cli

Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, *67*(1), 1–48. https://doi.org/10.18637/jss.v067.i01

Bowers, J., & Voors, M. (2016). How to improve your relationship with your future self. *Revista de Ciencia Política*, *36*(3), 829–848. https://doi.org/10.4067/S0718-090X2016000300011

Knuth, D. E. (1984). Literate programming. *The Computer Journal*, *27*(2), 97–111. https://doi.org/10.1093/comjnl/27.2.97

Nagler, J. (1995). Coding style and good computing practices. *PS: Political Science and Politics*, *28*(3), 488–492. https://doi.org/10.2307/420315

Nuijten, M. B., Hartgerink, C. H. J., Assen, M. A. L. M. van, Epskamp, S., & Wicherts, J. M. (2016). The prevalence of statistical reporting errors in psychology (1985–2013). *Behavior Research Methods*, *48*, 1205–1226. https://doi.org/10.3758/s13428-015-0664-2

Palleschi, D., Ronderos, C. R., & Knoeferle, P. (2025). Living in the present – how referent lifetime influences processing of past, present (perfect), and future tenses. *Glossa Psycholinguistics*, *4*(1), 1–48. https://doi.org/https://doi.org/10.5070/G601119481

Wilkinson, M. D. et al. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, *3*, 160018. https://doi.org/10.1038/sdata.2016.18

Xie, Y., Allaire, J. J., & Grolemund, G. (2018). *R markdown: The definitive guide*. Chapman; Hall/CRC. https://bookdown.org/yihui/rmarkdown

Xie, Y., Dervieux, C., & Riederer, E. (2020). *R markdown cookbook*. Chapman; Hall/CRC. https://bookdown.org/yihui/rmarkdown-cookbook

References

Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., & Dervieux, C. (2024). *Quarto*. https://github.com/quarto-dev/quarto-cli

Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, *67*(1), 1–48. https://doi.org/10.18637/jss.v067.i01

Bowers, J., & Voors, M. (2016). How to improve your relationship with your future self. *Revista de Ciencia Política*, *36*(3), 829–848. https://doi.org/10.4067/S0718-090X2016000300011

Knuth, D. E. (1984). Literate programming. *The Computer Journal*, *27*(2), 97–111. https://doi.org/10.1093/comjnl/27.2.97

Nagler, J. (1995). Coding style and good computing practices. *PS: Political Science and Politics*, *28*(3), 488–492. https://doi.org/10.2307/420315

Nuijten, M. B., Hartgerink, C. H. J., Assen, M. A. L. M. van, Epskamp, S., & Wicherts, J. M. (2016). The prevalence of statistical reporting errors in psychology (1985–2013). *Behavior Research Methods*, *48*, 1205–1226. https://doi.org/10.3758/s13428-015-0664-2

Palleschi, D., Ronderos, C. R., & Knoeferle, P. (2025). Living in the present – how referent lifetime influences processing of past, present (perfect), and future tenses. *Glossa Psycholinguistics*, *4*(1), 1–48. https://doi.org/https://doi.org/10.5070/G601119481

Wilkinson, M. D. et al. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, *3*, 160018. https://doi.org/10.1038/sdata.2016.18

Xie, Y., Allaire, J. J., & Grolemund, G. (2018). *R markdown: The definitive guide*. Chapman; Hall/CRC. https://bookdown.org/yihui/rmarkdown

Xie, Y., Dervieux, C., & Riederer, E. (2020). *R markdown cookbook*. Chapman; Hall/CRC. https://bookdown.org/yihui/rmarkdown-cookbook